

Software Engineering

@author Yangxin Wu

Last edited: 20251127

- [软件开发过程](#)
 - [两种基本模型](#)
 - [传统软件开发模型](#)
 - [流行和软件开发过程模型](#)
- [软件构造的目标](#)
- [软件测试](#)
 - [白盒测试](#)
 - [黑盒测试](#)
 - [单元测试](#)
 - [压力测试](#)
 - [性能测试](#)
 - [代码覆盖率测试](#)
 - [代码质量](#)

软件开发过程

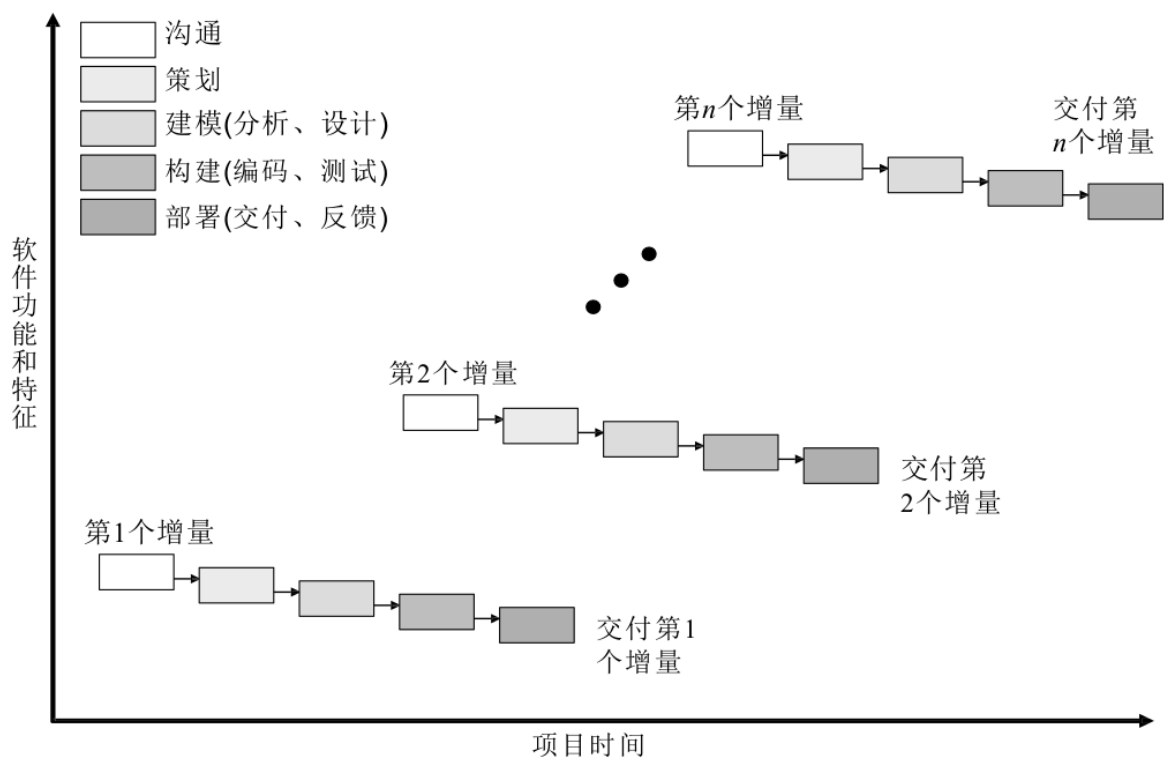
两种基本模型

- 线性模型
- 迭代模型

传统软件开发模型

- 瀑布模型
按照「需求分析 → 设计 → 编码 → 集成 → 测试 → 维护」的步骤顺序进行。管理简单，但无法适应需求的增加和变化。

- 增量过程



- 原型过程

快速地建立一个可以运行的程序，实现一部分功能。可以用来获得用户的真正需求，同时用来确定技术、成本和进度的可能性。

流行和软件开发过程模型

- 敏捷开发

通过快速迭代和小规模的持续改进来适应快速的变化。

- 测试驱动的开发

先写测试，然后编写能使测试通过的代码。「红灯 → 绿灯 → 重构」过程。

软件构造的目标

- 可理解性
- 可维护性
- 可复用性
- 时空性能

软件测试

测试的分类：

- 单元测试：对应编码阶段，其测试对象是单个模块或组件；
- 集成测试：对应详细设计，其测试对象是一组模块或组件；
- 系统测试：对应概要设计，其测试对象是整个系统；
- 验收测试：对应需求阶段，其测试对象是整个系统；

测试用例是为某个特殊目标而编制的一组测试输入、执行条件以及预期结果，用于核实是否满足某个特定软件需求。

测试用例的设计方法主要有黑盒测试法和白盒测试法。

- 黑盒测试：也称功能测试，着眼于程序外部结构，不考虑内部逻辑结构，主要针对软件界面和软件功能进行测试。
- 白盒测试：也称结构测试、透明盒测试、逻辑驱动测试或基于代码的测试，全面了解程序内部逻辑结构、对所有逻辑路径进行测试。

白盒测试

基于程序的源代码，已知产品的内部工作过程，主要是对程序内部结构展开测试，关注程序实现细节，检验程序中的每条通路是否都有按照预定要求正确工作。

黑盒测试

着眼于程序外部结构，在完全不考虑程序的内部逻辑结构和内部特性的情况下，测试者在程序接口进行测试，它只检查程序功能是否按照需求规格说明书的规定正常使用，程序是否能适当的接收输入数据而产生正确的输出信息。

单元测试

代码层面的测试，是对软件中的最小可测试单元进行检查和验证。

压力测试

测试在一定的负载下系统长时间运行的稳定性，但是这个负载不一定是应用系统本身造成的。

性能测试

主要测试使用时系统是否满足要求，同时可能为了保留系统的扩展空间而进行的一些稍稍超过“正常”范围的测试。

代码覆盖率测试

- 语句覆盖
至少执行程序中的所有语句一次。语句覆盖是最弱的逻辑覆盖准则。
- 判定覆盖
至少执行程序中每个分支一次，保证程序中每个判定节点取得每种可能的结果至少一次。
- 条件覆盖
不仅每个语句至少执行一次，而且使判定表达式中的每个条件都取到各种可能的结果（真、假）。
- 路径覆盖
使程序中每一条可能的路径至少执行一次。该策略是最强的，但一般不可实现。

代码质量

- 可维护性

在不破坏原有代码设计、不引入新的 **bug** 的情况下，是否能够快速地修改或者添加代码。

- 可读性

代码是否符合编码规范、命名是否达意、注释是否详尽、函数是否长短合适、模块划分是否清晰、是否符合高内聚低耦合等。

- 可扩展性

在不修改或少量修改原有代码的情况下，是否可以通过扩展的方式添加新的功能代码。

- 可复用性

应该编写可重用的、通用的代码，复用已有的代码。

- 可测试性

是否易于针对代码编写单元测试。

- 简洁性

尽量保持代码简单、逻辑清晰。